$ ~ AWS / LINUX / DEVOPS

Cloud & DevOps Command Reference

A Daily-Use Field Guide for Cloud & DevOps Engineers

Linux fundamentals | AWS CLI | Git | Docker | Kubernetes
Terraform | Ansible | CI/CD | Networking & Troubleshooting

Varun Kumar
AWS Solutions Architect | Cloud & DevOps Engineer | Team Lead
AWS Certified Solutions Architect - Linux Essentials - Generative AI
Pursuing: GCP Associate Cloud Engineer (ACE) | AWS AIF-C01

Table of Contents

Organized by problem domain, not alphabetically — find the right command by what you're trying to do.

1. Linux Navigation & File System	pwd, ls, cd, find, locate
2. File Inspection & Text Processing	cat, less, grep, awk, sed, cut, sort
3. Permissions & Ownership	chmod, chown, umask, ACLs
4. Process & Resource Management	ps, top, htop, kill, systemctl, jobs
5. Disk, Memory & System Info	df, du, free, uname, lscpu
6. Networking & Connectivity	ping, curl, wget, ss, netstat, dig, ssh, scp
7. Archiving & Compression	tar, zip, gzip, rsync
8. Package & Service Management	apt, yum, systemctl, journalctl
9. Git & Version Control	clone, branch, merge, rebase, stash
10. Docker	build, run, exec, logs, compose, prune
11. Kubernetes (kubectl)	get, describe, logs, exec, apply, rollout
12. AWS CLI — Core Services	EC2, S3, IAM, VPC
13. AWS CLI — DevOps & Automation	CloudWatch, Lambda, ECS, CloudFormation, boto3
14. Terraform	init, plan, apply, state, workspace
15. CI/CD & Automation (Jenkins, Ansible)	playbooks, pipelines, webhooks
16. Debugging Cheat Sheet	error → command → fix, quick lookup table

1. LINUX NAVIGATION & FILE SYSTEM

Use these to figure out **where you are** and **what exists** before doing anything else. Always your first move when debugging.

Command	What it does
`pwd`	Print current working directory (full absolute path)
`ls -la`	List all files incl. hidden, with permissions, owner, size, date
`ls -lh`	Human-readable file sizes (KB/MB/GB instead of bytes)
`ls -d */`	List only directory names (not files) in current path
`cd -`	Jump back to the previous directory you were in
`cd ~ or cd`	Go to home directory
`find / -name "file.txt"`	Search entire filesystem for a file by name
`find . -type f -mtime -1`	Find files modified in the last 1 day
`find . -type d`	Find only directories under current path
`find . -size +100M`	Find files larger than 100MB
`locate filename`	Fast file search using a pre-built index (needs updatedb)
`which python3`	Show the full path of an executable in your $PATH
`type ls`	Show whether a command is a builtin, alias, or binary
`realpath file.txt`	Resolve the absolute path of a relative path/symlink
`tree -L 2`	Show directory structure as a tree, 2 levels deep

TIP: Combine **`find`** with **`-exec`** to act on results directly: **`find . -name '*.log' -mtime +7 -exec rm {} \;`** deletes log files older than 7 days.

2. FILE INSPECTION & TEXT PROCESSING

This is the category you'll use most for **parsing logs, configs, and CLI output** — critical for DevOps work.

Viewing Files

Command	What it does
`cat file.txt`	Print entire file content to terminal
`less file.txt`	View file page-by-page (q to quit, / to search)
`head -n 20 file.txt`	Show first 20 lines
`tail -n 50 file.txt`	Show last 50 lines
`tail -f /var/log/app.log`	Follow a log file live as it's written (real-time)
`wc -l file.txt`	Count number of lines in a file
`diff file1.txt file2.txt`	Show differences between two files

Searching & Filtering (grep)

Command	What it does
`grep "ERROR" app.log`	Find lines containing 'ERROR'
`grep -i "error" app.log`	Case-insensitive search
`grep -r "TODO" ./src`	Recursively search all files in a directory
`grep -v "DEBUG" app.log`	Invert match — show lines NOT containing 'DEBUG'
`grep -n "timeout" app.log`	Show line numbers along with matches
`grep -c "ERROR" app.log`	Count matching lines instead of printing them
`grep -E "error|fail" app.log`	Extended regex — match multiple patterns
`grep -A 3 -B 3 "Exception" app.log`	Show 3 lines After and Before each match (context)

Text Transformation (awk, sed, cut)

Command	What it does
`cut -d',' -f1 data.csv`	Extract 1st column from comma-delimited file

Command	What it does
`awk '{print $1}' file.txt`	Print 1st whitespace-separated column
`awk -F: '{print $1}' /etc/passwd`	Print 1st field using ':' as delimiter
`awk '/ERROR/ {print $0}' app.log`	Print full lines matching pattern 'ERROR'
`sed 's/old/new/g' file.txt`	Replace all occurrences of 'old' with 'new'
`sed -n '5,10p' file.txt`	Print only lines 5 through 10
`sed -i 's/foo/bar/g' file.txt`	Edit file in-place (overwrites original)
`sort file.txt`	Sort lines alphabetically
`sort -n file.txt`	Sort lines numerically
`sort file.txt | uniq -c`	Count occurrences of each unique line
`sort -k2 -n data.csv | tail -5`	Top 5 rows by 2nd column (numeric)

TIP: Most useful one-liner for log debugging: `grep -i error app.log | awk '{print $1,$2}' | sort | uniq -c | sort -rn` — finds error lines, extracts timestamp, counts frequency, sorts by most common.

3. PERMISSIONS & OWNERSHIP

The #1 source of '**permission denied**' errors in scripts, deployments, and CI/CD pipelines.

Command	What it does
`ls -l file.sh`	Show permissions (rwx), owner, and group
`chmod +x script.sh`	Make a file executable
`chmod 755 script.sh`	Set exact permissions: owner=rwx, group/other=r-x
`chmod 644 file.txt`	Owner read/write, group & others read-only
`chmod -R 755 ./folder`	Apply permissions recursively to all files/subfolders
`chown user:group file.txt`	Change file owner and group
`chown -R ec2-user:ec2-user /app`	Recursively change ownership (common on EC2)
`umask`	Show default permission mask for new files
`sudo -l`	List what commands current user can run with sudo
`id`	Show current user's UID, GID, and group memberships
`getfacl file.txt`	View detailed Access Control List permissions

Reading chmod numbers fast

Number	Permission	Symbol
`7`	read + write + execute	`rwx`
`6`	read + write	`rw-`
`5`	read + execute	`r-x`
`4`	read only	`r--`
`0`	no permissions	`---`

A permission like **755** reads as three digits: owner, group, others. So 755 = owner has 7 (rwx), group has 5 (r-x), others have 5 (r-x).

4. PROCESS & RESOURCE MANAGEMENT

For when something is hanging, eating CPU, or you need to manage a long-running service/script.

Command	What it does
`ps aux`	List all running processes with user, CPU%, MEM%
`ps aux | grep nginx`	Find a specific process by name
`top`	Live, interactive view of processes (q to quit)
`htop`	Improved, color version of top (if installed)
`kill`	Gracefully terminate a process by its ID (SIGTERM)
`kill -9`	Force kill a process immediately (SIGKILL)
`pkill -f "process_name"`	Kill process by matching its name/command
`nohup ./script.sh &`	Run a script in background, immune to terminal close
`jobs`	List background jobs in current shell session
`fg %1`	Bring background job 1 to foreground
`disown -a`	Detach all background jobs from current shell
`systemctl status nginx`	Check status of a system service
`systemctl restart nginx`	Restart a service
`systemctl enable nginx`	Enable a service to start on boot
`journalctl -u nginx -f`	Live-tail logs for a specific systemd service

TIP: If a script seems frozen: open a second terminal and run **`ps aux | grep script_name`** to confirm it's actually running, then check what it's waiting on with **`strace -p <PID>`** (if available).

5. DISK, MEMORY & SYSTEM INFO

Use when an EC2 instance, container, or server is misbehaving due to resource exhaustion — extremely common root cause.

Command	What it does
`df -h`	Show disk space usage for all mounted filesystems (human-readable)
`du -sh ./folder`	Show total size of a specific folder
`du -sh * | sort -rh | head -10`	Find the 10 largest items in current directory
`free -h`	Show RAM and swap usage (human-readable)
`uname -a`	Show kernel version, hostname, architecture
`lscpu`	Show CPU architecture, core count, model info
`uptime`	Show how long system has been running + load average
`vmstat 2 5`	Show CPU/memory/IO stats, every 2s, 5 times
`iostat -x 2`	Show disk I/O statistics (needs sysstat package)
`hostname -I`	Show the server's IP address(es)
`lsblk`	List block devices (disks, partitions, mounted volumes)

TIP: On EC2, 'disk full' is one of the most common causes of app crashes. Run `df -h` first, then `du -sh /var/log/*` — runaway log files are usually the culprit.

6. NETWORKING & CONNECTIVITY

Essential for diagnosing connectivity issues between services, EC2 instances, VPCs, and external APIs.

Command	What it does
`ping google.com`	Check basic network connectivity / latency
`curl -I https://api.example.com`	Fetch only HTTP headers (quick health check)
`curl -v https://api.example.com`	Verbose request — shows full handshake/headers
`curl -X POST -d '{...}' url`	Send a POST request with JSON body
`wget https://file-url.com/file.zip`	Download a file from a URL
`ss -tulpn`	Show all listening ports and the process using them
`netstat -tulpn`	Older equivalent of ss (may need net-tools installed)
`dig example.com`	DNS lookup — full details (authoritative servers, TTL)
`nslookup example.com`	Simple DNS lookup
`traceroute example.com`	Show the network hops to reach a destination
`telnet host 22`	Test if a specific port is open/reachable
`nc -zv host 443`	Quick port-scan check using netcat
`ssh -i key.pem user@ip`	Connect to a remote server via SSH using a key
`scp -i key.pem file.txt user@ip:/path`	Copy a file to a remote server over SSH
`rsync -avz ./local/ user@ip:/remote/`	Sync directories efficiently (only changed files)

TIP: Can't reach an app on EC2? Check in this order: 1) **`ping`** the host, 2) **`ss -tulpn`** on the server to confirm the app is actually listening, 3) check the Security Group / NACL in AWS console, 4) **`curl -v`** from outside.

7. ARCHIVING & COMPRESSION

Command	What it does
`tar -czvf archive.tar.gz ./folder`	Create a compressed tarball (gzip)
`tar -xzvf archive.tar.gz`	Extract a .tar.gz archive
`tar -xzvf archive.tar.gz -C /target`	Extract into a specific target directory
`tar -tzvf archive.tar.gz`	List contents of archive without extracting
`zip -r archive.zip ./folder`	Create a .zip archive recursively
`unzip archive.zip`	Extract a .zip archive
`unzip -l archive.zip`	List zip contents without extracting
`gzip file.txt`	Compress a single file to file.txt.gz
`gunzip file.txt.gz`	Decompress a .gz file

tar flag meanings: `-c` create, `-x` extract, `-z` gzip, `-v` verbose, `-f` filename (always last before the filename).

8. PACKAGE & SERVICE MANAGEMENT

Command	What it does
`sudo apt update`	Refresh package index (Debian/Ubuntu)
`sudo apt install nginx`	Install a package (Debian/Ubuntu)
`sudo apt remove nginx`	Remove a package, keep config files
`sudo apt autoremove`	Remove unused dependency packages
`sudo yum install nginx`	Install a package (RHEL/CentOS/Amazon Linux 2)
`sudo dnf install nginx`	Install a package (newer RHEL/Amazon Linux 2023)
`dpkg -l | grep nginx`	Check if a package is installed (Debian-based)
`rpm -qa | grep nginx`	Check if a package is installed (RHEL-based)
`which nginx`	Confirm the binary exists and show its path

TIP: Amazon Linux 2 uses **`yum`**; Amazon Linux 2023 uses **`dnf`** (yum still works as an alias). Ubuntu EC2 AMIs use **`apt`**.

9. GIT & VERSION CONTROL

Everyday Workflow

Command	What it does
`git status`	Show changed/staged/untracked files
`git add .`	Stage all changes
`git commit -m "message"`	Commit staged changes with a message
`git push origin main`	Push commits to remote branch
`git pull origin main`	Fetch + merge latest changes from remote
`git log --oneline -10`	Compact log of last 10 commits
`git diff`	Show unstaged changes line by line
`git diff --staged`	Show staged changes before commit

Branching & Merging

Command	What it does
`git branch -a`	List all branches (local + remote)
`git checkout -b feature/new`	Create and switch to a new branch
`git switch feature/new`	Switch to an existing branch (modern alt to checkout)
`git merge feature/new`	Merge a branch into the current branch
`git rebase main`	Replay current branch commits on top of main
`git branch -d feature/new`	Delete a local branch (safe, only if merged)
`git push origin --delete feature/new`	Delete a remote branch

Undo / Fix Mistakes

Command	What it does
`git stash`	Temporarily shelve uncommitted changes
`git stash pop`	Reapply the most recent stash
`git reset --soft HEAD~1`	Undo last commit, keep changes staged
`git reset --hard HEAD~1`	Undo last commit, DISCARD all changes (destructive)

Command	What it does
`git revert`	Create a new commit that undoes a specific commit (safe for shared branches)
`git checkout -- file.txt`	Discard unstaged changes to a single file
`git cherry-pick`	Apply a specific commit from another branch

WARNING: Use `git revert` instead of `git reset --hard` on any branch others are pulling from — reset rewrites history and breaks teammates' local repos.

10. DOCKER

Images & Containers

Command	What it does
`docker build -t myapp:1.0 .`	Build an image from Dockerfile in current dir
`docker images`	List all local images
`docker run -d -p 8080:80 myapp`	Run container in background, map host:container port
`docker run -it myapp /bin/bash`	Run container interactively with a shell
`docker ps`	List running containers
`docker ps -a`	List all containers, including stopped ones
`docker stop`	Gracefully stop a running container
`docker rm`	Remove a stopped container
`docker rmi`	Remove an image

Debugging & Maintenance

Command	What it does
`docker logs`	View container's stdout/stderr logs
`docker logs -f`	Follow logs live
`docker exec -it bash`	Open a shell inside a running container
`docker inspect`	Full JSON metadata (network, mounts, env vars)
`docker stats`	Live CPU/memory usage of all running containers
`docker system prune -a`	Remove all unused images, containers, networks (frees disk space)
`docker network ls`	List Docker networks
`docker volume ls`	List Docker volumes

Docker Compose

Command	What it does
`docker compose up -d`	Start all services defined in docker-compose.yml, detached
`docker compose down`	Stop and remove all services + default network
`docker compose logs -f`	Follow logs from all services

Command	What it does
`docker compose ps`	List running services in this compose project
`docker compose restart`	Restart a single service

TIP: Container exits immediately after **`docker run`**? Check the real reason with **`docker logs <container_id>`** — almost always shows the crash error from inside the app.

11. KUBERNETES (kubectl)

Viewing Resources

Command	What it does
`kubectl get pods`	List pods in current namespace
`kubectl get pods -A`	List pods across ALL namespaces
`kubectl get pods -o wide`	List pods with node, IP, extra detail
`kubectl get svc`	List services
`kubectl get deployments`	List deployments
`kubectl get nodes`	List cluster nodes and their status
`kubectl describe pod`	Full details + recent events for a pod (key debug command)

Debugging

Command	What it does
`kubectl logs`	View logs from a pod
`kubectl logs -f`	Follow logs live
`kubectl logs -c`	Logs from a specific container in a multi-container pod
`kubectl logs --previous`	Logs from the PREVIOUS crashed instance of a pod
`kubectl exec -it -- bash`	Open a shell inside a running pod
`kubectl get events --sort-by=.lastTimestamp`	Cluster-wide events, most recent last
`kubectl top pods`	Show CPU/memory usage per pod (needs metrics-server)

Applying & Managing Changes

Command	What it does
`kubectl apply -f deployment.yaml`	Create/update resources from a YAML file
`kubectl delete -f deployment.yaml`	Delete resources defined in a YAML file
`kubectl rollout status deployment/myapp`	Watch a deployment rollout progress
`kubectl rollout undo deployment/myapp`	Roll back to the previous deployment version
`kubectl scale deployment myapp --replicas=5`	Manually scale a deployment

Command	What it does
`kubectl set image deployment/myapp app=myapp:v2`	Update container image (triggers rolling update)
`kubectl edit deployment myapp`	Edit a live resource directly in your default editor

TIP: Pod stuck in `CrashLoopBackOff`? Run `kubectl describe pod` first (check Events at the bottom), then `kubectl logs --previous` to see why the last attempt died.

12. AWS CLI — CORE SERVICES

EC2

`aws ec2 describe-instances`
List all EC2 instances with full metadata

`aws ec2 describe-instances --query "Reservations[].Instances[].[InstanceId,State.Name]" --output table`
List instance IDs + state in a clean table

`aws ec2 start-instances --instance-ids i-xxxx`
Start a stopped instance

`aws ec2 stop-instances --instance-ids i-xxxx`
Stop a running instance

`aws ec2 reboot-instances --instance-ids i-xxxx`
Reboot an instance

`aws ec2 describe-security-groups --group-ids sg-xxxx`
Show inbound/outbound rules for a security group

`aws ec2 describe-instance-status --instance-ids i-xxxx`
Check instance + system health status checks

S3

`aws s3 ls`
List all S3 buckets

`aws s3 ls s3://my-bucket/`
List objects inside a bucket

`aws s3 cp file.txt s3://my-bucket/`
Upload a file to S3

`aws s3 cp s3://my-bucket/file.txt .`
Download a file from S3

`aws s3 sync ./local-folder s3://my-bucket/`
Sync a local folder to S3 (only changed files)

`aws s3 rm s3://my-bucket/file.txt`
Delete a single object

`aws s3 rb s3://my-bucket --force`
Delete a bucket and all its contents

`aws s3api get-bucket-policy --bucket my-bucket`
View the bucket's resource policy (JSON)

IAM

`aws iam list-users`
List all IAM users in the account

`aws iam list-roles`
List all IAM roles

`aws iam get-user`
Show details of the currently authenticated IAM user

`aws iam list-attached-user-policies --user-name varun`
List policies attached to a specific user

`aws sts get-caller-identity`
Confirm WHO you're authenticated as right now (account, ARN)

`aws iam simulate-principal-policy --policy-source-arn --action-names s3:GetObject`
Test if a policy allows a specific action

VPC & Networking

`aws ec2 describe-vpcs`
List all VPCs in the region

`aws ec2 describe-subnets`
List all subnets

`aws ec2 describe-route-tables`
Show routing rules per subnet

`aws ec2 describe-nat-gateways`
List NAT gateways

`aws ec2 describe-internet-gateways`
List internet gateways

TIP: **`aws sts get-caller-identity`** is your fastest sanity check — run it whenever AWS CLI commands give unexpected 'Access Denied' errors, to confirm you're using the credentials/profile you think you are.

13. AWS CLI — DEVOPS & AUTOMATION

CloudWatch (Logs & Metrics)

`aws logs describe-log-groups`
List all CloudWatch log groups

`aws logs tail /aws/lambda/myFunction --follow`
Live-tail a log group (great for Lambda debugging)

`aws cloudwatch get-metric-statistics --namespace AWS/EC2 --metric-name CPUUtilization ...`
Pull a specific metric's data points

`aws cloudwatch describe-alarms`
List all CloudWatch alarms and their states

Lambda

`aws lambda list-functions`
List all Lambda functions

`aws lambda invoke --function-name myFn out.json`
Invoke a function synchronously, save output

`aws lambda update-function-code --function-name myFn --zip-file fileb://code.zip`
Deploy new code to an existing function

`aws lambda get-function --function-name myFn`
Get configuration + code location of a function

ECS / Containers

`aws ecs list-clusters`
List ECS clusters

`aws ecs list-tasks --cluster my-cluster`
List running tasks in a cluster

`aws ecs describe-services --cluster my-cluster --services my-svc`
Get service details + deployment status

`aws ecs update-service --cluster my-cluster --service my-svc --force-new-deployment`
Force a redeploy of a service

CloudFormation

`aws cloudformation deploy --template-file template.yaml --stack-name my-stack`
Deploy/update a CloudFormation stack

`aws cloudformation describe-stacks --stack-name my-stack`
Get stack status and outputs

`aws cloudformation describe-stack-events --stack-name my-stack`
See what happened during a deploy (great for debugging failed stacks)

`aws cloudformation delete-stack --stack-name my-stack`
Delete a stack and all its resources

boto3 (Python SDK) — quick patterns

```
import boto3

# EC2: list running instances
ec2 = boto3.client('ec2')
resp = ec2.describe_instances(
    Filters=[{'Name': 'instance-state-name', 'Values': ['running']}]
)

# S3: upload a file
s3 = boto3.client('s3')
s3.upload_file('local.txt', 'my-bucket', 'remote.txt')

# CloudWatch: get a metric
cw = boto3.client('cloudwatch')
cw.get_metric_statistics(
    Namespace='AWS/EC2', MetricName='CPUUtilization',
    Dimensions=[{'Name': 'InstanceId', 'Value': 'i-xxxx'}],
    StartTime=start, EndTime=end, Period=300, Statistics=['Average']
)
```

14. TERRAFORM

Command	What it does
`terraform init`	Initialize working dir, download providers/modules
`terraform plan`	Preview changes before applying (dry-run)
`terraform apply`	Apply changes to reach desired infrastructure state
`terraform apply -auto-approve`	Apply without interactive confirmation (use carefully)
`terraform destroy`	Tear down all resources managed by this state
`terraform validate`	Check configuration syntax without touching infra
`terraform fmt`	Auto-format .tf files to canonical style
`terraform show`	Show the current state in human-readable form
`terraform state list`	List all resources tracked in current state
`terraform state show`	Show detailed state of a single resource
`terraform import`	Bring an existing resource under Terraform management
`terraform workspace list`	List available workspaces (for managing multiple envs)
`terraform workspace select prod`	Switch to a different workspace (e.g., prod, staging)
`terraform output`	Show output values defined in the configuration

WARNING: Always run **`terraform plan`** and actually read the diff before **`apply`** — especially watch for lines marked '-/+' (destroy and recreate), which can cause unwanted downtime.

15. CI/CD & AUTOMATION (Jenkins, Ansible)

Ansible

Command	What it does
`ansible all -m ping -i inventory.ini`	Test connectivity to all hosts in inventory
`ansible-playbook deploy.yml -i inventory.ini`	Run a playbook against an inventory
`ansible-playbook deploy.yml --check`	Dry-run a playbook (shows changes without applying)
`ansible-playbook deploy.yml --limit webservers`	Run playbook against a specific host group only
`ansible-playbook deploy.yml -vvv`	Verbose output for debugging playbook failures
`ansible-vault encrypt secrets.yml`	Encrypt a file containing sensitive variables
`ansible-galaxy install role_name`	Install a role from Ansible Galaxy

Jenkins (CLI & common patterns)

`java -jar jenkins-cli.jar -s http://host build job-name`
Trigger a Jenkins job remotely

`curl -X POST JENKINS_URL/job/myjob/build --user user:token`
Trigger a build via REST API

`git log -1 --format=%H`
Get latest commit hash (often used in Jenkinsfile)

```
// Minimal Jenkinsfile pattern (Declarative Pipeline)
pipeline {
    agent any
    stages {
        stage('Build') {
            steps { sh 'docker build -t myapp:${BUILD_NUMBER} .' }
        }
        stage('Test') {
            steps { sh 'pytest tests/' }
        }
        stage('Deploy') {
            steps { sh 'kubectl apply -f k8s/deployment.yaml' }
        }
    }
}
```

16. DEBUGGING CHEAT SHEET

Quick lookup: **symptom** → **likely cause** → **command to confirm/fix.** This is the table to scan first when something breaks.

Symptom	Diagnose with
`"command not found"`	which / echo $PATH
`"permission denied"`	ls -l file / chmod +x file
`"no such file or directory"`	pwd / ls -la / find . -name file
`Script hangs / never finishes`	bash -x script.sh / ps aux | grep script
`"syntax error near unexpected token"`	bash -n script.sh (syntax check only)
`App can't connect to DB/service`	ping host / nc -zv host port / ss -tulpn
`EC2 instance unreachable`	Check Security Group / aws ec2 describe-instance-status
`Disk full / app crashing randomly`	df -h / du -sh /var/log/*
`Container exits immediately`	docker logs
`Pod CrashLoopBackOff`	kubectl describe pod / kubectl logs --previous
`"Access Denied" on AWS CLI`	aws sts get-caller-identity / check IAM policy
`Terraform plan shows unwanted destroy`	terraform state show / review .tf diff carefully
`High server load / slow app`	top or htop / uptime / vmstat 2 5
`Git push rejected (non-fast-forward)`	git pull --rebase origin main
`Variable empty in script when it shouldn't be`	echo "var=[$var]" / set -u to catch unset vars

Universal debug workflow for any shell script

```
#!/bin/bash
set -e            # exit on first error
set -u            # error on undefined variables
set -o pipefail   # catch failures inside pipes

# run with: bash -x script.sh   to trace every line live
```

Compiled for daily AWS / Linux / DevOps reference. Pair this with hands-on practice — run each command in a sandbox/EC2 instance to build muscle memory.